

LAB 7: Infrastructure as Code

In this lab, we are learning how to use IaC tools like Terraform to create and manage infrastructure in AWS.

Part 1: Create simple EC2 instance on AWS

1. Install Terraform CLI according to your OS
<https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli>
2. Create a directory terraform-lab to store your Terraform configuration files. This will be your working directory for this lab.
3. Create terraform.tf file in terraform-lab and add terraform configuration

Terraform relies on plugins called providers to interact with cloud providers, SaaS providers, and other APIs. Terraform configurations must declare which providers they require so that Terraform can install and use them. Additionally, some providers require configuration (like endpoint URLs or cloud regions) before they can be used.

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 4.16"  
    }  
  }  
  required_version = ">= 1.2.0"  
}
```

```
provider "aws" {  
  region = "us-west-2"  
}
```

4. Create main.tf file in terraform-lab to provision a **simple EC2 instance** on AWS using Terraform.

```
resource "aws_instance" "app_server" {  
  ami      = "ami-830c94e3"  
  instance_type = "t2.micro"
```

```
tags = {
  Name = "ExampleAppServerInstance"
}
```

5. Initialize the project by running on your terminal `terraform init`
6. Preview the infrastructure by running on your terminal `terraform plan`
7. Provision the infrastructure by running `terraform apply`

Submission: After running terraform apply, you will see terraform.tfstate file generated. Explain what is terraform state file.

Part 2: Infrastructure Change

Infrastructure is continuously evolving and Terraform helps you manage that change. As you change Terraform configurations, Terraform builds an execution plan that only modifies what is necessary to reach your desired state.

8. Update your ami in your main.tf file to `ami-08d70e59c07c61a3a`
9. Run terraform apply to apply the changes.

Part 3: Input Variable

The current configuration includes a number of hard-coded values. Terraform variables allow you to write configuration that is flexible and easier to re-use.

10. Add a variable to define the instance name. Create a new file called `variables.tf` with a block defining a new `instance_name` variable.

```
variable "instance_name" {
  description = "Value of the Name tag for the EC2 instance"
  type       = string
  default    = "ExampleAppServerInstance"
}
```

11. In `main.tf`, update the `aws_instance` resource block to use the new variable.

```
tags = {
  Name = var.instance_name
}
```

12. Run terraform apply and you will see that there is no change as we just only parameterized the variable.
13. Change the default value of variable *instance_name* and run *terraform apply* then you will the infrastructure change. *At the same time, you can run terraform apply -var "instance_name=YetAnotherName"*

14. Create a file *outputs.tf* and run terraform apply then *terraform output*

```
output "instance_id" {  
    description = "ID of the EC2 instance"  
    value      = aws_instance.app_server.id  
}
```

```
output "instance_public_ip" {  
    description = "Public IP address of the EC2 instance"  
    value      = aws_instance.app_server.public_ip  
}
```

15. Follow Hasicorp to learn how to store remote state
<https://developer.hashicorp.com/terraform/tutorials/aws-get-started/aws-remote>
16. Run terraform destroy command terminates resources managed by your Terraform project.

Submission:

- Screenshot important steps to get the output
- Explain what a Terraform remote state file is and why you should store Terraform state remotely.